WEEK – 02

Multi Level Queue:

```
#include <stdio.h>

struct process {
    int id, arrival, burst;
    int start, completion;
    int waiting, turnaround, response;
};

void fcfs(struct process p[], int n) {
    int current_time = 0;
    for(int i=0; i<n; i++) {
        if(current_time < p[i].arrival)
            current_time = p[i].arrival;
        p[i].start = current_time;
        current_time += p[i].burst;
        p[i].completion = current_time;
        p[i].turnaround = p[i].completion - p[i].arrival;
        p[i].waiting = p[i].turnaround - p[i].burst;
        p[i].response = p[i].start - p[i].arrival;
    }
}

void display(struct process p[], int n){
    printf("\n===== GANTT CHART =====\n|");
    for(int i=0;i<n;i++){
        printf(" P%d |", p[i].id);
    }
    printf("\n0");
    for(int i=0;i<n;i++){
        printf("   %d", p[i].completion);
    }
    printf("\n");

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    float totalTAT=0, totalWT=0, totalRT=0;
    for(int i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
            p[i].id, p[i].arrival, p[i].burst,
            p[i].completion, p[i].turnaround,
            p[i].waiting, p[i].response);
        totalTAT += p[i].turnaround;
        totalWT += p[i].waiting;
        totalRT += p[i].response;
    }
    printf("\nAverage TAT = %.2f\n", totalTAT/n);
    printf("Average WT  = %.2f\n", totalWT/n);
    printf("Average RT  = %.2f\n", totalRT/n);
}

int main(){
    int n;
    printf("Enter number of processes: ");
    scanf("%d",&n);

    struct process p[n];
    int type;

    for(int i=0;i<n;i++){
        p[i].id = i+1;
        printf("\nProcess P%d\n", p[i].id);
        printf("Enter Arrival Time: "); scanf("%d",&p[i].arrival);
        printf("Enter Burst Time: "); scanf("%d",&p[i].burst);
        printf("Enter Queue (1=RR,2=FCFS,3=SJF): "); scanf("%d",&type);
    }

    fcfs(p,n);
    display(p,n);

    return 0;
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL                po

PS C:\Users\SHALINI K A\Desktop\OS> gcc MultiLevelQueue.c -o MultiLevelQueue
PS C:\Users\SHALINI K A\Desktop\OS>
PS C:\Users\SHALINI K A\Desktop\OS> ./MultiLevelQueue
Enter number of processes: 3

Process P1
Enter Arrival Time: 0
Enter Burst Time: 5
Enter Queue (1=RR,2=FCFS,3=SJF): 1

Process P2
Enter Arrival Time: 1
Enter Burst Time: 4
Enter Queue (1=RR,2=FCFS,3=SJF): 2

Process P3
Enter Arrival Time: 2
Enter Burst Time: 3
Enter Queue (1=RR,2=FCFS,3=SJF): 3

===== GANTT CHART =====
| P1 | P2 | P3 |
0   5   9   12

PID     AT      BT      CT      TAT     WT      RT
P1      0       5       5       5       0       0
P2      1       4       9       8       4       4
P3      2       3       12      10      7       7

Average TAT = 7.67
Average WT  = 3.67
Average RT  = 3.67
```

LAB PROGRAM-2

2. Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the process in the system are divided into 2 categories.

```
# Include <stdio.h>
struct Process {
    int Pid, arrival, burst;
    int start, completion;
    int waiting, turnaround, response;
};
void bcbs (struct process P[], int n){
    int current_time=0;
    for(int i=0; i<n; i++){
        if(current_time < P[i].arrival)
            current_time = P[i].arrival;
        P[i].start = current_time;
        current_time += P[i].burst;
        P[i].completion = current_time;
        P[i].turnaround = P[i].completion -
                          P[i].arrival;
        P[i].waiting = P[i].turnaround -
                       P[i].burst;
        P[i].response = P[i].start - P[i].arrival;
    }
}
void display (struct process P[], int n){
    printf("\n ===Gantt chart===\n");
    for(int i=0; i<n; i++){
        printf(" P%d", P[i].pid);
    }
```

```
    printf("\n 0");
    for(int i=0; i<n; i++){
        printf(" %d", P[i].completion);
    }
    printf("\n");
    printf("\n PID\t AT\t BT\t CT\t TAT\t WT\t
           RT\n");
    float totalTAT=0, totalWT=0, totalRT=0;
    for(int i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d
               \t%d\n",
            P[i].pid, P[i].arrival, P[i].burst,
            P[i].completion, P[i].turnaround,
            P[i].waiting, P[i].response);
        totalTAT += P[i].turnaround;
        totalWT += P[i].waiting;
        totalRT += P[i].response;
    }
    printf("\nAverage TAT = %.2f\n", totalTAT/n);
    printf("Average WT = %.2f\n", totalWT/n);
    printf("Average RT = %.2f\n", totalRT/n);
}
int main(){
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    struct process P[n];
    int type;
    for(int i=0; i<n; i++){
        P[i].pid = i+1;
        printf("\n Process P%d\n", P[i].pid);
        printf("Enter arrival time: ");
        scanf("%d", &P[i].arrival);
```

```
        printf("Enter Burst time: ");
        scanf("%d", &P[i].burst);
        printf("Enter queue (1=RR, 2=FCFS, 3=SJF):");
        scanf("%d", &type);
    }
    bcbs(P, n);
    display(P, n);
    return 0;
}
```

Output:

```
Enter number of Process: 3
Process P1
Enter Arrival time: 0
Enter Burst time: 5
Enter queue (1=RR, 2=FCFS, 3=SJF): 1

Process P2
Enter Arrival time: 1
Enter Burst time: 4
Enter queue (1=RR, 2=FCFS, 3=SJF): 2

Process P3
Enter Arrival time: 2
Enter Burst time: 3
Enter queue (1=RR, 2=FCFS, 3=SJF): 3
```

Gantt Chart

P1	P2	P3

0 5 9 12

PID	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	1	4	9	8	4	4
P3	2	3	12	10	7	7

Average TAT = 7.67
Average WT = 3.67
Average RT = 3.67